EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 11 — 26/11/2025

Author: Jonathan Dutoit

Professor: Maria Brbic

Handling text data (Part 2)

Revisiting the 4 typical tasks

- Document retrieval
- Document classification
- Sentiment analysis
- Topic detection

Bag of words matrix is composed of docs **rows** and words as **columns**

- TF-IDF matrix to the rescue:
  - Entry for doc d, word w: $\text{tf}(d, w) \cdot \text{idf}(w)$
    Where tf is the term frequency and idf is the inverse document frequency. It gives the word frequency and it's weight based on how many times it appears in the document and cross-documents! Words like "the", "is", "a" are down-weighted, but words like "perception" are up-weighted.

Document retrieval

Typical task 1

- Nearest-neighbor method in spirit of kNN
- Compare query doc q to all documents in the collection (i.e., rows of the TF-IDF matrix)
- Rank docs from collection in increasing order of distance
- Distance metrics
  - Typically cosine distance (= 1 - cosine similarity)
  - Recall: cosine similarity of q and $v = < \frac{q}{|q|}, \frac{v}{|v|} >$
  - If rows are L2-normalized, may simply take dot products $< q, v >$

  - By using the angle as distance metric, the size of document doesn't matter anymore. This was an issue found with Euclidean distance.

Document retrieval

- This is just the most basic approach
- For efficiency
  - Start by filtering documents by presence of query terms (use efficient full-text index)
  - Hugely narrows down set of documents to be ranked
- Google et al. do much more...
  - Query-independent relevance: PageRank
  - Boost recent results
  - Personalization, contextualization
  - ...

Document classification

Typical task 2

- Use TF-IDF matrix as feature matrix for supervised methods
- Often more features (words) than documents
- What's the danger with this? **Undefined system of equations, a.k.a over-fitting, or high variance**
- Potential solutions:
  - Use more data (i.e., more labeled training docs)
  - Decrease model capacity:
    * Feature selection
    * Regularization
    * Dimensionality reduction
  - Use ensemble methods such as random forests

Sentiment analysis

Typical task 3

- When treated as classification, we can do the same as previously
- When treated as regression: pretty much the same (most supervised methods work for both classification and regression)

Regularization

- E.g., linear regression:
  Find weight vector β that minimizes $\sum_{i=1}^{n}(y_i - \mathbf{x_i}^T\beta)^2$
- Remedy: penalize very large positive and very large negative weights:

$$\text{minimize } \sum_{i=1}^{n}(y_i - \mathbf{x_i}^T\beta)^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

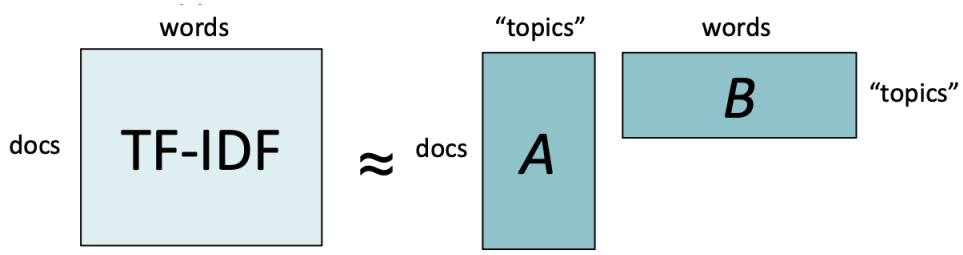
  For one word j that has very small training error on the training data which was made by making β_j large enough.

Topic detection

Typical task 4

- Cluster rows of TF-IDF matrix (each row a data point)
- Manually inspect clusters and label them with descriptive names(e.g., "news", "sports", "romance", "tech", "politics")
- In principle, may use k-means, k-medoids, etc.
- But can be difficult if dimensionality is large (#words >> #docs)
  - "Curse of dimensionality"
  - Many outliers

Alternative approach: **matrix factorization**



- Assume docs and words have representation in (latent) "topic space"
- (IDF-weighted) word frequency modeled as dot products of doc's vectors and word's vectors in topic space
- #topics << #words (⇒ "dimensionality reduction"): D*W ⇒ (D+W)*T
- Topics interpretable in doc space (A's cols) and word space (B's rows)
- Optimization problem:
  - Find A, B such that AB is as close to TF-IDF matrix as possible. That is, minimize

$$\sum_{d=1}^{N} \sum_{w=1}^{M} (T_{dw} - A_d B_w)^2$$

where T is TF-IDF matrix, A_d is d-th row of A, and B_w is the w-th column of B

- This is called **latent semantic analysis (LSA)**

You already know how to efficiently compute this, from your linear algebra class: **singular-value decomposition** (SVD)

- $T = USV^T$, where U and V have columns that are orthogonal to each other (orthonormal bases), and S is diagonal with in decreasing order. S captures "importance" of topic (amount of variation in corpus w.r.t. topic)
- If you want k topics, keep only the first k columns of U and V, and the first k rows and columns of $S \Rightarrow U'$, S', V'
- E.g., $A = U', B = S'V'^T$ or $A = U'S', B = V'^T$

Recall potential problem with clustering and classification and regression: "curse of dimensionality". **Matrix factorization** via **LSA** solves these problems for you:

- Use A instead of original TF-IDF matrix
- That is, cluster (or learn to classify or regress) in topic space, rather than word space

Topic representation from LSA is simply a vector, not a probability distribution over topics. **Probabilistic**: LDA = Latent Dirichlet Allocation

LDA: probabilistic topic modeling

- Latent Dirichlet Allocation
- Document := bag of words
- Topic := probability distribution over words
- Each documents has a (latent) distribution over topics
- "Generative story" for generating a doc of length n:
  d := sample a topic distribution for the doc ($\Leftarrow$ "Dirichlet")
  for $i = 1, ..., n$

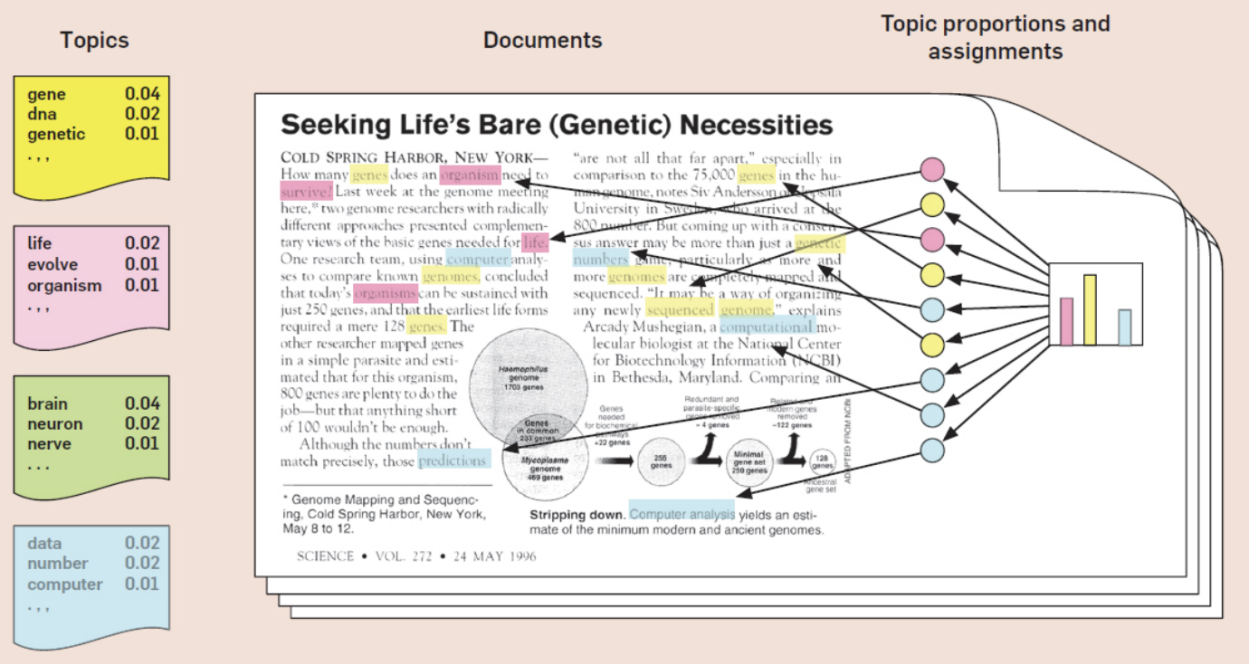
  t := sample a topic from topic distribution d

  w := sample a word from topic t

  Add w to the bag of words of the doc to be generated

This is **maximum-likelihood inference problem**!

LDA example



- Each topic is represented as a distribution probability over words! We have, in order: **genetics**, **evolution**, **neuroscience** and **computer science**.
- The bar chart on the right is a categorical distribution over the select points. We can sample from this distribution all generated words!
- LDA just gives the multi-set of words, not the order. The idea of LDA is to revert the above described process!

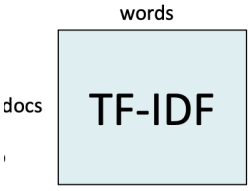
Topic inference in LDA

- LDA is unsupervised (topics come out "magically")
- Input:
  - Docs represented as bags of words
  - Number K of topics
- Output:
  - K topics (distributions over words)
  - For each doc: distribution over K topics $\Rightarrow \theta_1 = \{$genetics: 0.3, evolution: 0.5, neuroscience: 0.2, computer science: 0$\}$
- How is this done?
  - Find distributions (i.e. topics, docs) that maximize the likelihood of the observed documents (maximum likelihood)

TF-IDF matrix

Question:

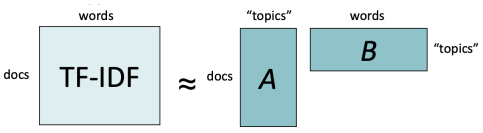
- "Which of these word pairs is more closely related?"
  - (car, bus)
  - (car, astronaut)
- How to quantify this?
- Detour:
  - How to quantify closeness of two docs?
  - E.g., cosine of **rows** of TF-IDF matrix



- Retour:
  - How to quantify closeness of two words?
  - E.g., cosine of **cols** of TF-IDF matrix. Cosine will be large if 2 words appears in the same set of documents!

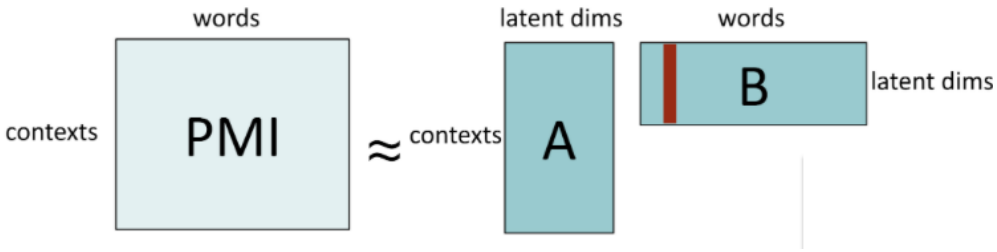
Sparsity in TF-IDF matrix

- Two docs (i.e., rows of TD-IDF matrix)
  - "Do you love men"?
  - "Adorest thou the likes of Adam?"
  - Cosine of row vectors of TF-IDF matrix == 0
- Same problem when comparing two words (i.e., cols of matrix)
- Solution:
  - Move from sparse to dense vectors. But how?
  - Latent semantic analysis (LSA)! Use columns of B as dense vectors representing words



"Word vectors"

- Idea: take into account how words appear together! Columns of TF-IDF matrix (sparse) or of word-by-topic matrix B (dense), but all word representation is an entire doc treated as bag of words. All information about word proximity, syntax, etc., is lost.
- Solution:
  - Instead of full docs, consider local context: windows of L (e.g., 3) consecutive words to left and right of the target word
  - Rows of matrix: not docs, but contexts.
  - Pointwise mutual information(PMI):
    $M[c,\ w] = \text{PMI}(c,\ w) = \log \frac{Pr(c,w)}{Pr(c) \cdot Pr(w)}$
    "How much more likely are c and w to occur together than if they were independent?"



- word2vec: factor PMI matrix and use columns of B as word vectors

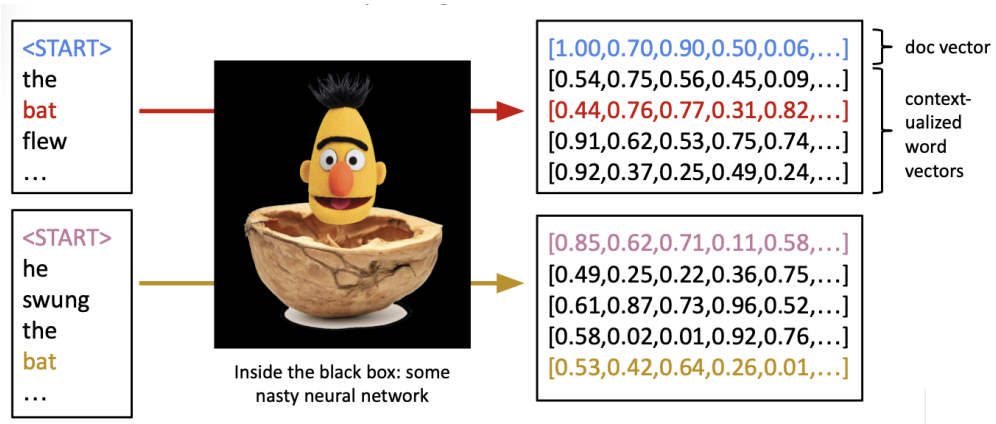
Beyond bags of words

From words to texts

- How to represent larger units, such as sentences, paragraphs, docs?
- Typical approach: take sum/average of words vectors
- Note: this is roughly also what bags of words are (when using "one-hot" encoding for words, i.e., vector with exactly one 1, rest 0)
- More recently: **learn** vectors for longer units
  - Convolutional neural networks
  - Recurrent neural networks: LSTM, ELMo
  - Transformed-based models: BERT, GPT-*

BERT in a nutshell

Introduced in 2018 by Google Research



NLP pipeline

- Tokenization
- Sentence splitting
- Part-of-speech (POS) tagging
- Named-entity recognition (NER)
- Coreference resolution
- Parsing
  - Shallow parsing (a.k.a chunking)
  - Constituency parsing
  - Dependency parsing
- **Sequential model**: fixed order of steps, early errors will propagate downstream. Fixer order not optimal for all cases...
- Hence, current research
- **Today's trend**: generative language models.